Detection of Sudoku Puzzle using Image Processing and Solving by Backtracking, Simulated Annealing and Genetic Algorithms: A Comparative Analysis

Snigdha Kamal[1], Simarpreet Singh Chawla[2], Nidhi Goel[3]
[1, 2, 3]Dept. of Electronics and Communication Engineering
Delhi Technological University (Formerly DCE)
New Delhi - 110042, India
[1]snigdhakamal17@gmail.com, [2]chawla.simarpreet@gmail.com, [3]nidhi.iitr@gmail.com

***Abstract*—In this paper, we propose the digital detection and decryption of a sudoku puzzle using vision based techniques and subsequent solving of the puzzle using three algorithms- Backtracking, Simulated Annealing and Genetic Algorithm. The proposed method can recognize any sudoku puzzle captured from a digital camera and after employing appropriate pre-processing algorithms which include adaptive thresholding, Hough Transform and geometric transformation, the digits are recognized using Optical Character Recognition (OCR), and based on their pixel locations in the image, they are stored in corresponding locations in the 9×9 matrix. The detected puzzles of varying complexity levels are then solved using the three algorithms and the results are compared and contrasted, indicating the relative efficiencies of the three techniques in accurately solving Sudoku puzzles. Simulated Annealing performed the best amongst the three algorithms, whereas, genetic algorithm performed the worst in the comparison.**

***Index Terms*—Sudoku, Image processing, Optical Character Recognition, Backtracking, Simulated Annealing, Genetic Algorithm**

I. Introduction

The Sudoku is a highly addictive, logic based puzzle involving combinatronics and number-positioning. The classic game consists of a 9×9 grid partially filled with numbers from 1 to 9. The target is to fill the puzzle in such a way that every column, row and 3×3 sub grid contains the numbers 1 to 9 without repetition or elimination. The fewer the number of entries filled at the beginning, the harder is to solve the puzzle. Moreover, a well-posed sudoku puzzle has only one solution. A sudoku puzzle can be considered as a binary satisfaction, graph colouring or an exact cover problem. Sudoku detectors and solvers have previously been implemented by employing dedicated hardware [1]–[3] based on genetic algorithms, simulated annealing and by using vision-based systems [4], [5].

In this paper, we present a quantitative comparison where the sudoku puzzle, after its detection using vision-based techniques is subsequently solved by employing three algorithms- Backtracking, Simulated Annealing and Genetic Algorithm. The results derived experimentally in terms of time taken to solve puzzles of varying difficulties are compared and contrasted for every algorithm, thus indicating the relative speed and efficiencies of the three procedures in solving sudoku puzzles.

II. Related Work

The sudoku puzzle being a number placement puzzle has been popular as a satisfiability research and constraint problem. Numerous algorithms and techniques have been proposed for identifying and solving it, both on hardware and software.

Literatures [1]–[3] have utilized different algorithms for solving sudoku puzzle and have implemented them on FPGA. Thirer [1] used adaptions regarding chromosome definitions, mutation and crossover phases, calculation of fitness function and population definitions for solving sudoku puzzle using genetic algorithm. Malakonakis *et al.* [2] made use of simulated annealing for solving a sudoku puzzle of the order 15×15×15×15, whereas, a brute force approach is employed to solve sudoku puzzle on FPGA in [3].

Some literatures have also applied the concept of image processing or entropy minimization to identify or solve sudoku puzzles [4], [5]. Wicht and Hennebert [4] detected line of the grid and characters of a sudoku puzzle by using Hough transform and Deep Belief Network (DBN) respectively. Simha *et al.* [5] both identified and solved the puzzles using template matching and backtracking algorithm respectively.

Many literatures have employed various algorithms for solving sudoku puzzle [6]–[11]. Wu and Lin [6] proposed a new algorithm called disjoint minimal unavoidable set (DMUS) and were able to reduce the time taken by a single core computer to solve a sudoku puzzle by a factor of 128.67. Both Mantere [7] and Pacurib *et al.* [8] employed an improved variant of artificial bee colony algorithm for solving sudoku. Moon *et al.* [9] used an Sinkhorn-based solver, whereas, Mantere and Koljonen [10] employed genetic algorithm for performing the tasks of generating, rating and solving sudoku puzzles. Gunther *et al.* [11] utilized the approach of entropy minimization for solving sudoku.

The proposed system, built for solving puzzles of standard size (order 3), detects sudoku puzzle using image processing techniques, which is followed by a quantitative comparison between three different algorithms- backtracking, simulated

978-1-5090-0148-4/15/$31.00© 2015 IEEE

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.

Fig. 1. Acquired Image of Sudoku Puzzle

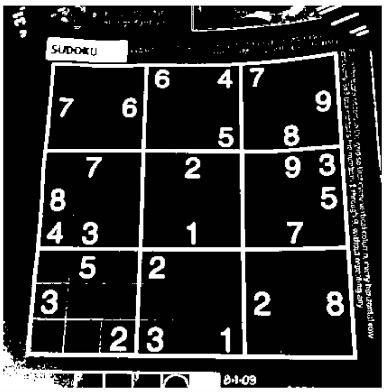

Fig. 2. Segmented Image After Local Thresholding

annealing and genetic algorithm in terms of time taken to solve sudoku puzzles of different complexities.

III. Detection of Sudoku Puzzle

The detection of Sudoku puzzle is achieved in three stages: First, the puzzle is segmented and extricated from its surrounding environment using local thresholding. Second, geometric transformation is applied to the image, which is thresholded again to extract the digits. Third, Optical Character Recognition(OCR) is used to extract the numbers from the puzzle which are stored in a 9x9 matrix according to their pixel locations in the image.

A. Image preprocessing and puzzle extraction

The first step involves image procurement and application of preprocessing techniques to take care of non-uniformity in illumination gradient, background noise and puzzle orientation as shown in Fig. 1.

The image I is acquired into the system (either from a camera or a pre-stored file) and converted to grayscale format. A local thresholding method, which takes into account the local contrast(due to background noise and non-uniform illumination) by using local descriptors like mean and standard deviation is then employed, which converts the image I into a binary image by calculating a local threshold at the central point (x, y) of every neighbourhood in I. An array of zeroes and ones defines the size of the neighbourhood, where ones indicate the elements used in the calculation of standard deviation and local mean. The thresholded binary image is then given by equation (1):

$$S(x,y) = \begin{cases} 1 & \text{for } I(x,y) > T_{xy} \\ 0 & \text{for } I(x,y) \leq T_{xy} \end{cases} \tag{1}$$

where $T_{xy} = a\sigma_{xy} + bm_G$

T_{xy} is local threshold

m_G is global image mean

σ_{xy} is local standard deviation

a and b are non-negative constants that have been established experimentally

Local thresholding effectively conserves the essential details, thus making our proposed system more efficient for number recognition as shown in Fig. 2.

The next step is to extract the puzzle from its background eliminating all the excessive elements. To achieve this, we used area as our primary criterion. All the connected components in the image are labeled, the component with the largest area(the outer box) is extracted.The boundary of the quadrilateral in the image is then detected using Hough Transformation and lines are plotted over the box. The intersection of the four lines gives the corners of the sudoku puzzle and these intersection

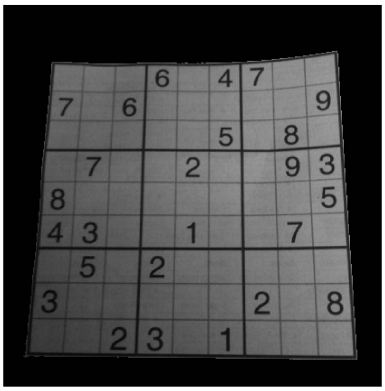

Fig. 3. Extracted Puzzle

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.

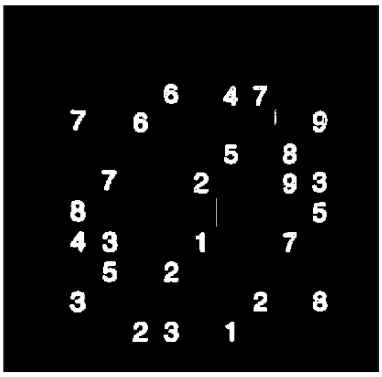

Fig. 4. Binary Image for number recognition

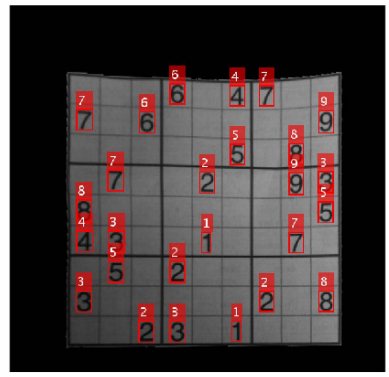

Fig. 5. Resultant Image after number recognition using OCR

points are reserved for further use.

The extracted box is used to generate a binary mask which when applied to the original Sudoku puzzle, obliterates the background clutter, leaving just the sudoku puzzle behind as shown in Fig. 3.

B. Image transformation and conversion to binary image

Geometric transformation of the extricated puzzle is done to account for any kind of perspective distortion or convexity defects and to resize the puzzle to a standard square of size 450×450 pixels. This scaling is brought about using projective transformation of the image where the four intersection points reserved in the previous step are mapped into the four corners of a pre-defined square. Thus, puzzles of various sizes are mapped into a uniform square for furthure processing, making our system robust.

The spatially transformed image is then converted into a binary image using adaptive thresholding and the boundary and grid lines are discarded which highlights the numbers required for recognition as shown in Fig. 4.

C. Number recognition using OCR

The Number Recognition is done in two stages:

1) Stage 1: The length and breadth of the square grid are divided into 9 equal parts yielding 81 smaller squares. The diagonal x and y co-ordinates of each small square are computed and re-assigned values from 1 to 9, specifying the row and column number of each square. Subsequently, the centroid values of each digit present in the puzzle are determined and these coordinate values are re-assigned values from 1 to 9 by repeatedly comparing the x and y coordinates of the centroid to the coordinates of each smaller square. If the centroid lies within the x and y coordinates of a square, it takes the value of the row and column number of that square.
2) Stage 2: The numbers present in the image are recognized and classified using OCR. The OCR yields highly accurate results under the condition that the noise present around the characters in the image is minimal. Therefore, it is crucial that the image processing stages for noise removal yield sharp and lucid images for OCR detection as shown in Fig. 4. The identified digits are stored in a 9×9 matrix based on their new centroid values and unoccupied places are assigned zeros in the matrix.

IV. Sudoku Solving Techniques

We have employed three different algorithms- *Backtracking*, *Simulated Annealing* and *Genetic Algorithm* for solving the sudoku puzzle. The backtracking algorithm iterates through all the possible solutions of sudoku, simulated annealing utilizes the randomization and swapping of numbers and the genetic algorithm mimics the working of natural evolution where the optimal solution is realized by an iterative process of selection and recombination.

A. Backtracking Algorithm

Backtracking algorithm solves constraint satisfaction problems by finding all or some of the solutions. Backtracking constructs some candidates for solving a problem, but abandons a candidate *i.e.* backtracks when the particular candidate cannot lead or be a valid solution. In the case of sudoku, backtracking is dependent upon the constraints defined by the user and how the partial candidates will lead into a complete valid solution. Moreover, backtracking will be able to solve every sudoku puzzle with a unique solution in a finite amount of time. Backtracking algorithm is explained in Algorithm 1.

The algorithm starts by finding an empty cell. If the empty cell is found, then every number possible is tried in that empty cell. If a number is safe to fill the cell, *i.e.*, does not violate row, column and block constraints, the the number is filled and algorithm then finds another empty cell. If no number could be filled without violating the constraints, then, the algorithm backtracks and fills the previously filled cell with another number or empties it. Hence, the backtracking

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Backtracking Algorithm

```
procedure BACKTRACKING
    Input: Sudoku Puzzle, Order of Sudoku (N)
    Function (Solve)
    if Find an empty cell then
        for Number = 1:N^2 do
            if Safe to fill the Number then
                Fill the empty cell with the number
                if (Solve) then
                    Output: Sudoku solved
                end if
                Empty the cell
            end if
        end for
        Return False
    else
        Output: Sudoku solved
    end if
end procedure
```

algorithm iterating through every possible solution is able to solve the sudoku puzzle.

B. Simulated Annealing Algorithm

Simulated annealing is based on the concept of annealing in metallurgy where metals are heated and then slowly cooled to decrease the defects and increase the strength of the metal. Unlike other optimization techniques,simulated annealing has the advantage of not getting stuck in local minima or maxima. Simulated Annealing algorithm is explained in Algorithm 2.

The simulated annealing algorithm starts with setting up the initial values of maximum temperature, minimum temperature, alpha, maximum number of iterations per temperature drop and random filling of the unfilled cells of the unsolved sudoku puzzle in such a way that each $N \times N$ block consists of every number ranging from 1 to N^2, where N is the order of the sudoku (N=3 in this case). Then, threats, *i.e.*, wrongness of the data placement, is computed using equation (2):

$$\sum threats = count \times (count - 1)/2 \tag{2}$$

where, $count$ is the instances of a number (or candidate) in a row or column. Therefore, total threats is the sum of threats of every number (or candidate) in every column and row. Furthermore, to change the initial position of the sudoku puzzle, two non-fixed cells are swapped randomly in a randomly chosen $N \times N$ block and the threats is re-calculated. If the new threats is less than the threats computed last and greater than $e^{(change\ in\ threats)/Temp_Max}$, then the altered sudoku puzzle is accepted or else the same swap (undo) is done again. The exponential function allows erroneous swaps in the beginning of the algorithm to ensure that simulated annealing algorithm doesn't get stuck at a local minimum. As the algorithm proceeds, it becomes greedier, *i.e.*, reduces the acceptance of erroneous swaps. Finally, when the threats becomes zero, the algorithm is exited.

Algorithm 2 Simulated Annealing

```
procedure SIMULATED ANNEALING
    Input: Sudoku Puzzle, Order of Sudoku (N)
    Defined: Max_Temp, Min_Temp, Max_Iterations, alpha
    Randomly fill each block using numbers 1:N^2
    Threat_Curr = Calculate Threats
    Threat_Best = Threat_Curr
    while Min_Temp < Max_Temp do
        i = 1
        while i < Max_Iterations do
            Switch two non-fixed numbers in a block
            Threat_i = Calculate Threats
            if !Threat_i then
                Output: Sudoku solved
                Exit
            end if
            if Threat_i < Threat_Curr then
                Threat_Curr = Threat_i
                if Threat_Curr < Threat_Best then
                    Threat_Best = Threat_Curr
                end if
            else if exp((Threat_Curr-Threat_i)/Max_Temp) > random(0,1) then
                Threat_Curr = Threat_i
            else
                Swap Back
            end if
            i = i + 1
        end while
        Max_Temp = Max_Temp * alpha
    end while
    Output: Sudoku not solved
end procedure
```

C. Genetic Algorithm

Genetic algorithms are based on the fundamental concepts of evolution in biology. The classic Genetic Algorithm starts with a population of randomly generated initial solutions or chromosomes encoded as bit arrays. A fitness function which determines the strength of the solution, *i.e.*, how well the solution fits within the constraints, is used to rank the initial bit arrays. The higher ranked arrays are selected for reproduction probabilistically, so that the lower-ranked solutions also have a small chance of reproduction by a process called crossover: the first part of a selected bit array is concatenated with the last part of the second selected bit array. A low level of mutation is then applied to the solution arrays by randomly flipping the solution bits. The bigger the population, the better the final solution due to an increase in genetic diversity. The new solution arrays, with the fittest arrays of the previous generation comprise the population of the new generation, and the process of survival and reproduction continues till an optimal solution has been generated. Genetic algorithm used

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUDOKU DETECTION RESULTS

Image Number	Runtime (sec)	Total Numbers Not Detected	Total Numbers Detected Incorrectly	Image Number	Runtime (sec)	Total Numbers Not Detected	Total Numbers Detected Incorrectly
1	1.888	0	1	16	1.250	0	0
2	2.408	0	0	17	1.604	0	0
3	3.827	2	0	18	1.222	0	0
4	2.575	0	0	19	1.273	0	0
5	2.554	1	0	20	1.510	0	0
6	2.858	3	0	21	1.278	0	0
7	2.585	4	1	22	1.298	0	0
8	2.790	2	1	23	1.302	0	0
9	2.814	2	0	24	2.183	0	0
10	2.531	0	0	25	1.459	0	0
11	2.103	0	0	26	1.326	0	0
12	1.421	0	0	27	1.245	0	0
13	1.158	0	0	28	1.157	0	0
14	1.138	0	1	29	1.151	0	0
15	1.151	0	0	30	1.341	0	0

Algorithm 3 Genetic Algorithm

```
procedure GENETIC ALGORITHM
    Input: Sudoku Puzzle, Order of Sudoku (N)
    Defined: Fitness Points, Penalty Points, Crossover
             Points, Population Size
    Create an initial population of 1-D solution arrays
    repeat
        Fitness function to determine fitness :
        if Sudoku rules withheld by individual then
            fitness++
        end if
        if Sudoku rules violated by individual then
            penalty++
        end if
        repeat
            Assign part of a roulette to each individual
            Select two individuals from the roulette
            Randomly choose a crossover point
            Concatenate individuals at crossover point
        until New population has been produced
    until Sudoku puzzle solved
    if Fitness level same for 15 generations then
        Restart process
    end if
end procedure
```

for solving sudoku puzzle is explained in Algorithm 3.

Each solution is represented by an array of integers with size equal to the number of unknowns in the Sudoku puzzle. A fitness function evaluates the fitness level of all solution arrays. Each individual is assigned a penalty point for each rule broken (each row, column and 3×3 block should contain numbers from 1 to 9 only once) and a fitness point for each rule it upholds. The roulette wheel selection mechanism is used to select individuals for recombination. A part of the roulette is assigned to each individual proportionate to its fitness level when compared with the net fitness of the population. Hence, all individuals have a possibility of being selected, but fitter individuals possess a higher probability of doing so. The two individuals selected are recombined in a fixed number of crossover points. This process of selection and recombination continues till a new generation is created. The new generation is again evaluated for fitness, and the process continues till an optimal solution is reached or the algorithm is stuck in a local optima. In the latter case, if the process is stuck at the same fitness level for 15 generations, the algorithm is re-started with a new set of random solution arrays.

V. RESULTS

The detection of sudoku puzzle was implemented using image processing techniques and solved using three different algorithms- Backtracking, Simulated Annealing and Genetic Algorithm.

We ran the proposed method for detecting sudoku on a dataset containing thirty sudoku puzzles filled with perspective distortions, various scales and ideal images. Table I presents the detailed analysis of the runtime and errors encountered in the process of detection of thirty sudoku puzzles. The method employed was able to accomplish 73.33% success rate on the data set. Moreover, 98.835% of the numbers were detected, out of which 99.424% of the numbers were correctly guessed. The average time taken to detect the sudoku puzzle was 1.813 seconds.

For solving the sudoku puzzle, three different algorithms were used. Sudoku puzzles of order 3 with complexity varying from 68 to 17 were solved using each of the three algorithms. The complexity of a sudoku is defined as the number of fixed numerals already present in an unsolved sudoku out of 81 (N = 3). The time taken by the three algorithms individually for solving sudoku puzzle of each complexity (68-17) is shown in Fig. 6, which also compares the performance of the three algorithms in terms of common logarithm ($\log_{10}$) of the time taken. The common logarithm of time taken ensures that minute details between the performance of the

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.

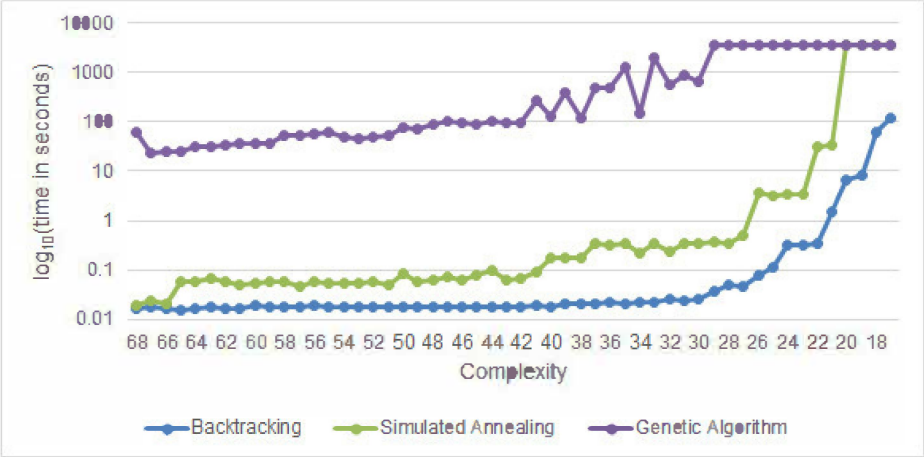

Fig. 6. Comparison Between Backtracking, Simulated Annealing and Genetic Algorithm

three algorithms can be observed. Furthermore, maximum time taken for solving a sudoku puzzle was fixed to 3600 seconds.

It can be inferred from Fig. 6 that as the complexity of the sudoku puzzle increases, the performance of all the three algorithm degrades. It can easily be deduced that genetic algorithm performs worse than simulated annealing and backtracking because genetic algorithm experiences slow convergence, due to which it loses the inability to escape from the local minima. Only by changing the fundamental parameters of the genetic algorithm or by using a larger population of solutions can lead to the solving of complex sudoku puzzles in less time. Moreover, while comparing simulated annealing and backtracking, it can be observed that there isn't much difference between the performance of the two algorithms, but when the complexity level is increased beyond 27, the performance of the simulated annealing degrades quickly. This can be attributed to increase in struggle by simulated annealing to not get stuck in a local minima. On the account of the fact that beyond the complexity of 27, the number of empty cells to be filled become greater than 54 and the number of local minima increase, hence requiring higher value of parameters like temperature and maximum iterations. Furthermore, backtracking had the best performance among all the algorithms for every value of complexity, but its performance also degrades for higher complexity due to the increase in unfilled cells, which further leads to higher instances of backtracking while solving a sudoku puzzle, thus, increasing the time taken to solve a sudoku puzzle.

VI. Conclusion

A robust sudoku detector and solver has been implemented and tested yielding fairly accurate solution data. The digits present in the puzzle were recognized with 98.835% accuracy, with the accuracy of numbers recognized correctly being 99.424%. The comparative analysis between the three algorithms employed for solving the sudoku puzzle showed that backtracking had the best performance amongst the three, while genetic algorithm performed the worst and was unable to solve standard sudoku puzzles of higher complexities under the set time limit. Moreover, the performance of simulated annealing whilst being better than genetic algorithm exponentially degraded when complexity of the sudoku puzzle exceeded 27. Therefore, the relative efficiencies of the three algorithms were studied experimentally and reported, yielding backtracking as the best algorithm to solve sudoku puzzles accurately in a short time frame.

References

[1] N. Thirer, "About the fpga implementation of a genetic algorithm for solving sudoku puzzles," in *Electrical & Electronics Engineers in Israel (IEEEI), 2012 IEEE 27th Convention of*, pp. 1–3, IEEE, 2012.

[2] P. Malakonakis, M. Smerdis, E. Sotiriades, and A. Dollas, "An fpga-based sudoku solver based on simulated annealing methods," in *Field-Programmable Technology, 2009. FPT 2009. International Conference on*, pp. 522–525, IEEE, 2009.

[3] K. Van der Bok, M. Taouil, P. Afratis, and I. Sourdis, "The tu delft sudoku solver on fpga," in *Field-Programmable Technology, 2009. FPT 2009. International Conference on*, pp. 526–529, IEEE, 2009.

[4] B. Wicht and J. Hennebert, "Camera-based sudoku recognition with deep belief network," in *Soft Computing and Pattern Recognition (SoCPaR), 2014 6th International Conference of*, pp. 83–88, IEEE, 2014.

[5] P. J. Simha, K. Suraj, and T. Ahobala, "Recognition of numbers and position using image processing techniques for solving sudoku puzzles," in *Advances in Engineering, Science and Management (ICAESM), 2012 International Conference on*, pp. 1–5, IEEE, 2012.

[6] H.-H. Lin, I. Wu, *et al.*, "Solving the minimum sudoku poblem," in *Technologies and Applications of Artificial Intelligence (TAAI), 2010 International Conference on*, pp. 456–461, IEEE, 2010.

[7] T. Mantere, "Improved ant colony genetic algorithm hybrid for sudoku solving," in *Information and Communication Technologies (WICT), 2013 Third World Congress on*, pp. 274–279, IEEE, 2013.

[8] J. Pacurib, G. M. M. Seno, J. P. T. Yusiong, *et al.*, "Solving sudoku puzzles using improved artificial bee colony algorithm," in *Innovative Computing, Information and Control (ICICIC), 2009 Fourth International Conference on*, pp. 885–888, IEEE, 2009.

[9] T. K. Moon, J. H. Gunther, and J. J. Kupin, "Sinkhorn solves sudoku," *Information Theory, IEEE Transactions on*, vol. 55, no. 4, pp. 1741–1746, 2009.

[10] T. Manter and J. Koljonen, "Solving, rating and generating sudoku puzzles with ga," in *Evolutionary Computation, 2007. CEC 2007. IEEE Congress on*, pp. 1382–1389, IEEE, 2007.

[11] J. Gunther and T. Moon, "Entropy minimization for solving sudoku," *Signal Processing, IEEE Transactions on*, vol. 60, no. 1, pp. 508–513, 2012.

Authorized licensed use limited to: Univ of Calif Davis. Downloaded on April 02,2026 at 21:52:00 UTC from IEEE Xplore. Restrictions apply.